

P r o x y

Proxy Pattern

A stand-in that controls access

DEFINITION

Provide a surrogate for another object to control access to it.

Proxy provides a surrogate with the same interface as its subject, so it intercepts every call and can guard, defer, cache, or relay before the real work. Callers can't tell the proxy from the real object — one shape serving four jobs: virtual (lazy), remote, protection, and smart reference.

WHY IT MATTERS

Sometimes you need a smarter reference than a plain pointer: defer creating an expensive object until it is used, check access rights, stand in for an object in another process, or add caching and ref-counting. A proxy adds that control behind the subject's own interface, transparently, without touching the real object. The costs: indirection and hidden latency — a remote proxy can block or throw where a local call wouldn't.

— How it works

Subject	The common interface shared by RealSubject and Proxy.
RealSubject	The real object the proxy represents and forwards to.
Proxy	Holds a reference, controls access, then delegates to RealSubject.

HOW TO APPLY

1. Have the Proxy implement the same Subject interface as the real object.
2. Hold a reference to the RealSubject (or create it lazily on first use).
3. Run the policy — guard, defer, cache — then forward the call.
4. Keep it transparent: honor the subject's full interface and semantics.

LITMUS TEST

Do you need to intercept access to an object — lazy-load, guard, cache, or relay it — without changing it or its callers? If yes, stand a proxy in front.

— Smell → fix

A costly object is created eagerly, even when it is never used:

```
class Report {  
  data = loadHugeDataset() // runs now, always  
}  
const r = new Report() // pays even if never drawn
```

↓ STAND A PROXY IN FRONT

```
interface Report { rows(): Row[] }  
class LazyReport implements Report {  
  #real?: Report  
  rows() { // create on first use, then reuse  
    return (this.#real ??= new RealReport()).rows()  
  }  
}
```

LazyReport shares Report's interface but builds the real object only on the first rows() call, then reuses it. The same idea swaps in guard / cache / remote policies — all behind the subject's interface, invisibly to callers. Mind hidden latency, and that a proxy is not == the real subject.

USE WHEN

- ✓ Lazy loading
- ✓ Access control, caching, remote stubs

AVOID WHEN

- ▲ No access concern — a plain reference is simpler

— Trade-offs & pitfalls

PROS

- + Adds control transparently
- + Lazy and cheap

CONS

- Indirection / latency
- Another layer to reason about

COMMON PITFALLS

- ▲ Keep it transparent: a proxy must honor the subject's full interface and semantics.
- ▲ Hidden latency / failure: a remote proxy can block or throw where a local call wouldn't — explicit async is often clearer.
- ▲ Identity and equality: proxy is not == realSubject; instanceof and == can mislead (and JS Proxy traps are subtle).

WHERE YOU SEE IT

- ▶ ORM lazy loading (Hibernate, TypeORM): entities are virtual proxies that hit the DB on first access.
- ▶ RPC / gRPC stubs (remote proxy); API gateways and CDNs (caching / protection proxies).
- ▶ The JS Proxy object trapping get / set — powers Vue and MobX reactivity and validation.

SEE ALSO

Decorator	same wrapping shape — but adds behavior rather than controlling access
Adapter	also wraps one object — but changes the interface instead of keeping it
Facade	simplifies a whole subsystem; a Proxy substitutes for one object
Flyweight	copy-on-write sharing is a smart-reference proxy over a shared object

— Interview & sources

Proxy vs Decorator?

Same wrapping structure; intent differs — Proxy controls access (lazy, remote, guard), Decorator adds responsibilities.

Name the four proxy kinds.

Virtual (lazy), Remote (cross-process), Protection (access control), Smart reference (ref-count / cache / lock).

How does ORM lazy loading use it?

Entities are virtual proxies; touching a lazy field triggers the DB fetch behind the same interface.

What is a JS Proxy?

A language-level proxy whose traps (get / set / has) intercept operations — it powers Vue and MobX reactivity.

REMEMBER

A stand-in with the subject's interface that controls access — lazy, remote, protection, or smart reference.

SOURCES

GoF — "Design Patterns" (1994): Proxy (structural)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Proxy